Parallel and Distributed Systems 

Pr. Olivier Gruber

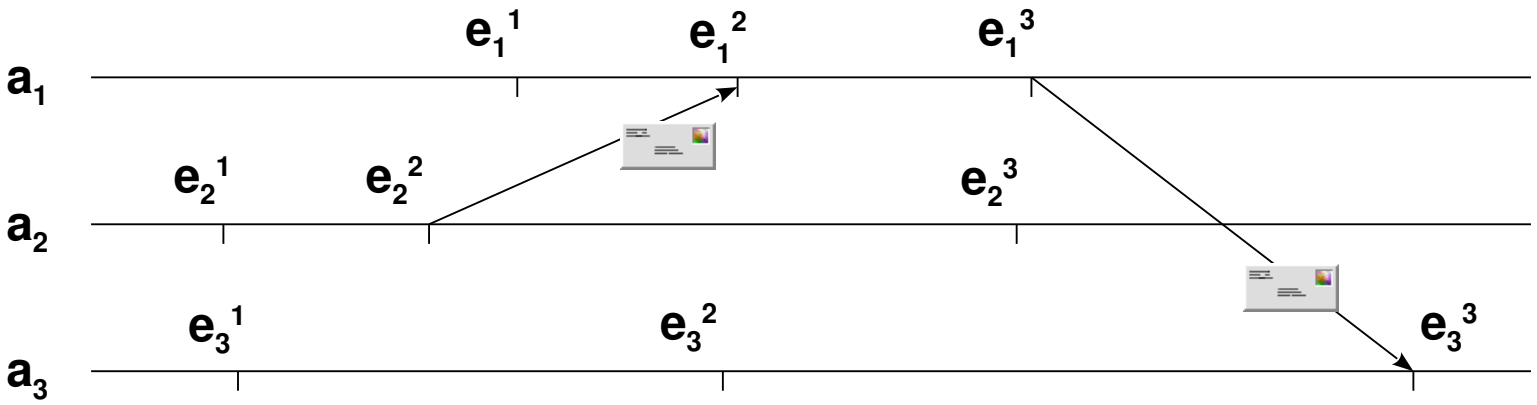
olivier.gruber@univ-grenoble-alpes.fr

Université Grenoble Alpes

©Pr. Olivier Gruber

Today

- Distributed Systems & Fault-tolerance
  - We assume an distributed system, sending and receiving messages
  - We assume a Perfect Failure Detector (PFD)
  - Looking at the impacts of considering failures on otherwise fairly simple algorithms…



©Pr. Olivier Gruber

About Failures – By Google

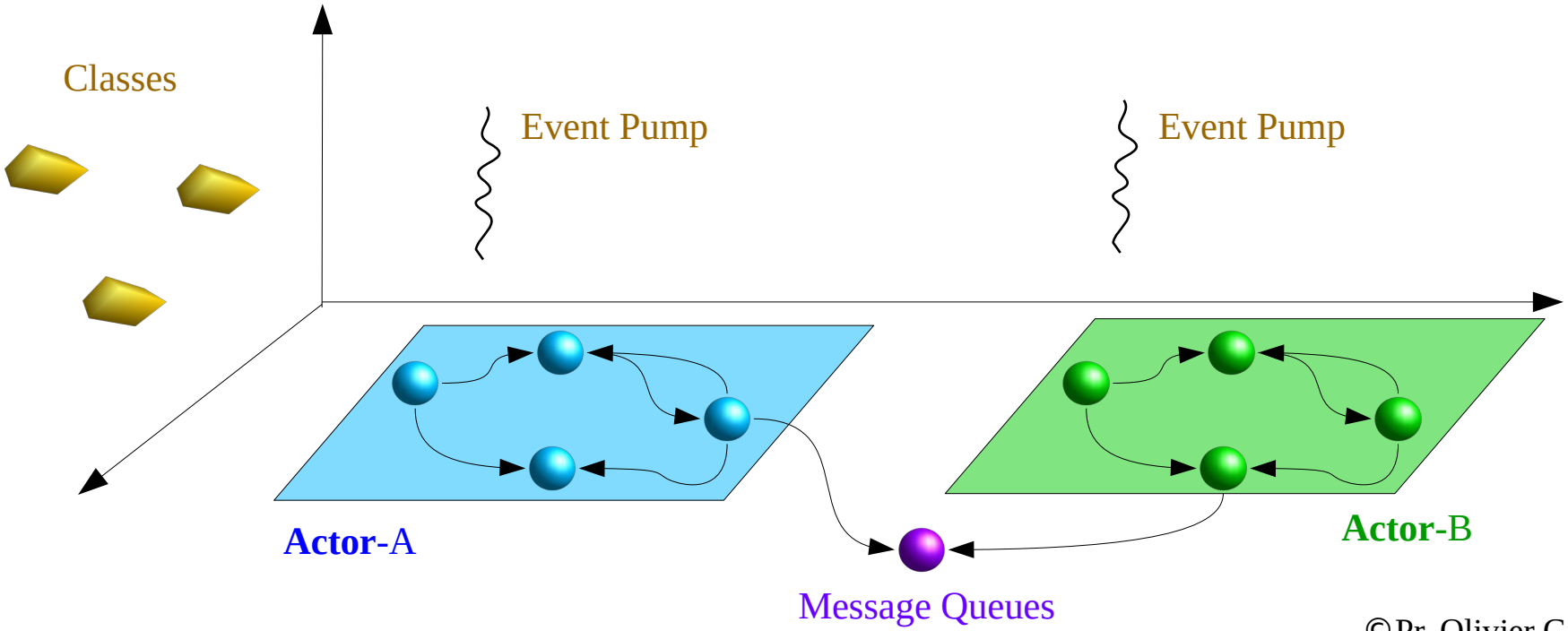
- Google fellow Jeff Dean in 2008 said:
  - Basic cluster of 1,800 servers, 40 to 80 servers per rack
  - Failure during the first year of each cluster's life
    - 1,000 individual machine will fail
    - Thousands of hard driver failures
    - One power distribution unit will fail, bring down 500 to 1000 machines for about 6 hours
    - 20 racks will fail, each time causing 40 to 80 machines to vanish from the network
    - 5 racks will "go wonky" with 50% network packet loss
    - The cluster will have to be rewired once, affecting 5% of the machines at any given moment over 2 days
    - 50% chance that the cluster will overheat, taking down most of the servers in less than 5mn, taking 1 to 2 days to recover

Source: https://www.datacenterknowledge.com/archives/2008/05/30/failure-rates-in-google-data-centers

©Pr. Olivier Gruber

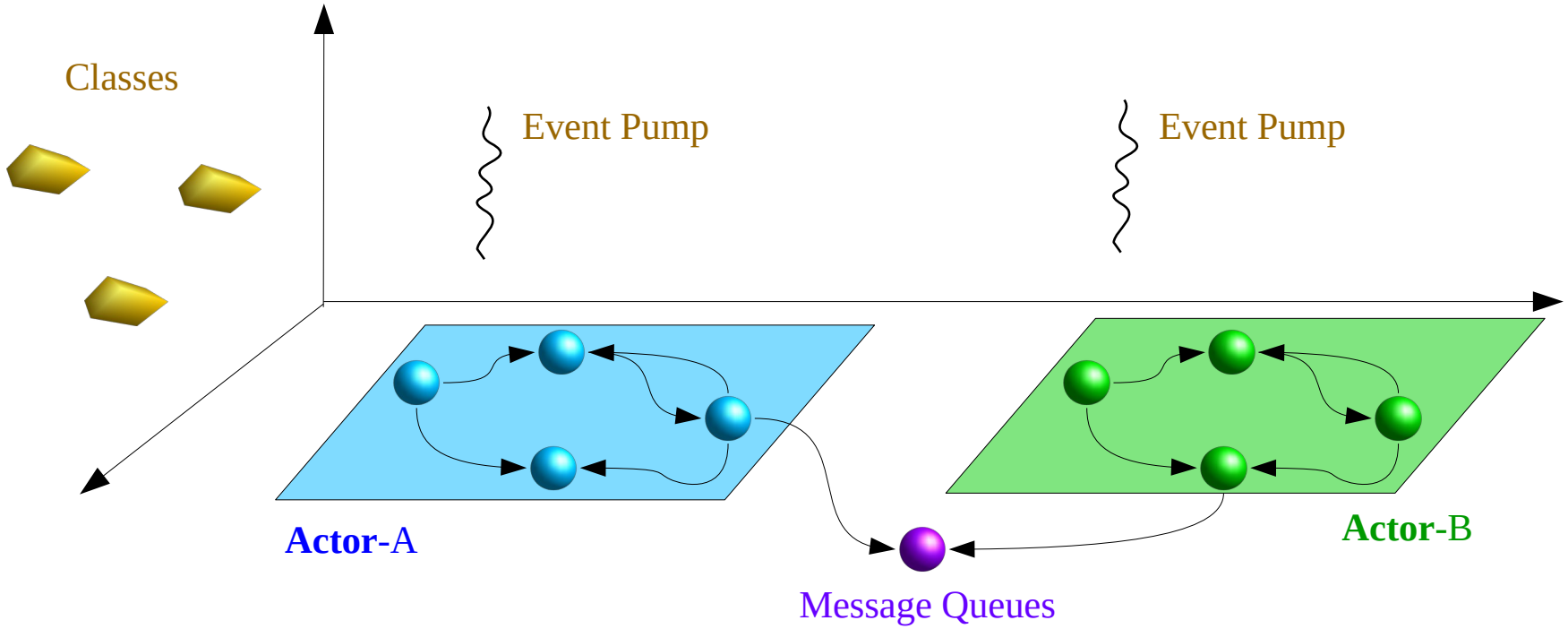
Actors and Message Queues

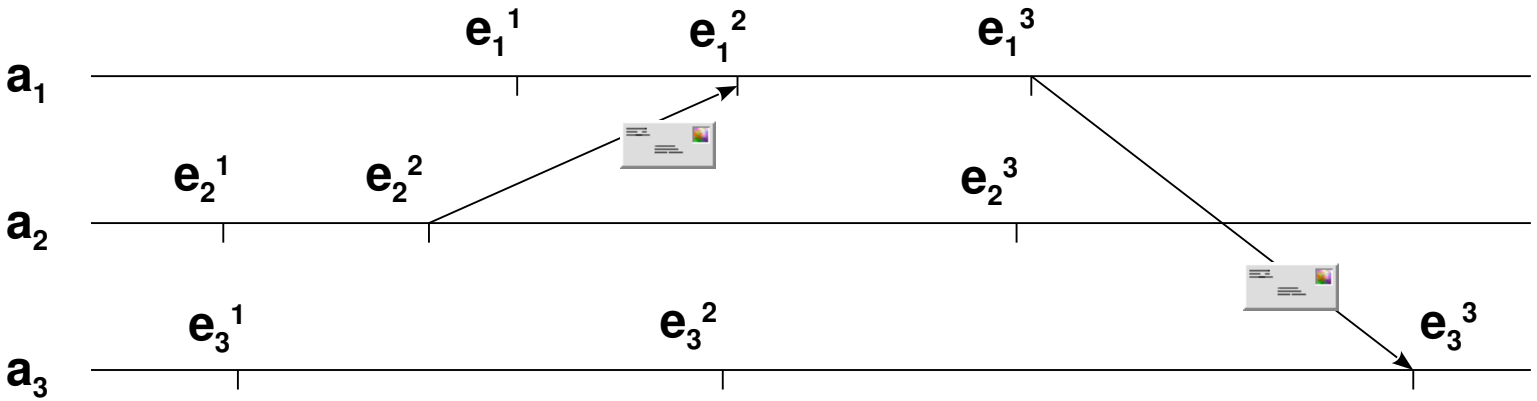
- Actor Model
  - An actor is active, it "*owns*" one or more tasks
  - An actor "*owns*" an isolated object graphs, except for aliased queues but not messages
  - Actors may choose to catch exceptions or not (death)
  - Upon its death, the state of an actor is lost, related queues are closed
  - Messages are either copied or migrated, reception is an event
  - Conceptually, each actor has its own event pump



©Pr. Olivier Gruber

Distributed Peers / Actors

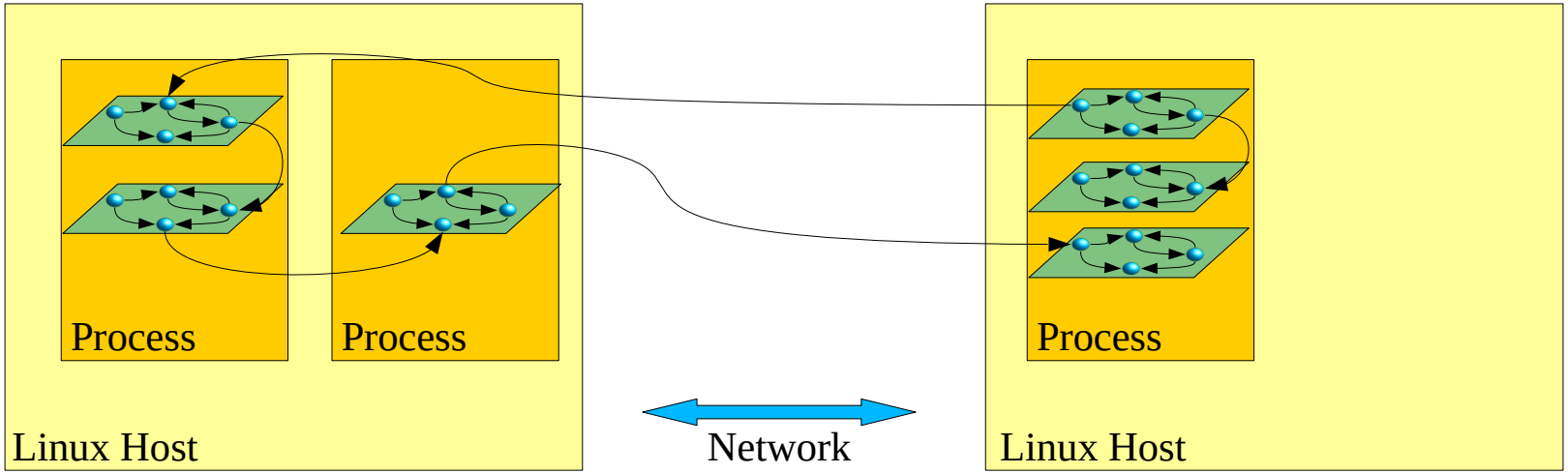




©Pr. Olivier Gruber

Distributed Actors – Processes & Network

- **Actors**, **events**, and **messages**
  - Each **actor** is a sequence of reactions to events (**event-oriented programming**)
  - **Messages** go through **FIFO and lossless communication channels**
  - **Receiving a message is an event**
  - Conceptually, **each actor has its own event pump**
  - **Actors may fail**



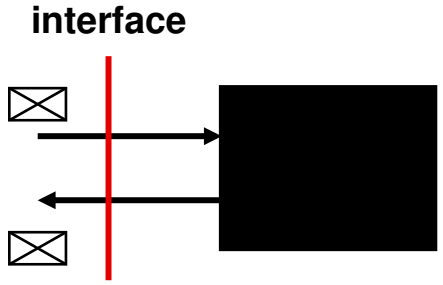
Note that a Linux process, executing a single program, respects the Actor Model

Note that for Java, a single Java runtime may host a single actor or several

©Pr. Olivier Gruber

Discussing Failures

- Fail-Stop processes
  - Each process is either **correct** or it is **failed**
  - When correct, it runs and works fine
  - When failed, it neither sends nor receives messages

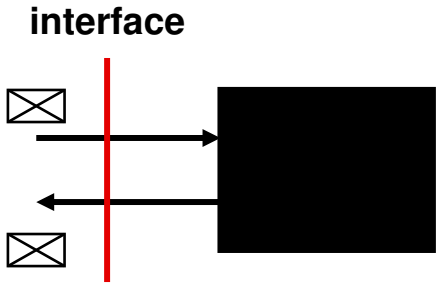


- Black-box model
  - Each process is considered a black-box from a failure model perspective
  - *A process has failed when it does not behave according to its specification*
  - This is not a precise definition, it is specification-dependent
  - This assumes that the specification is complete and correct

©Pr. Olivier Gruber

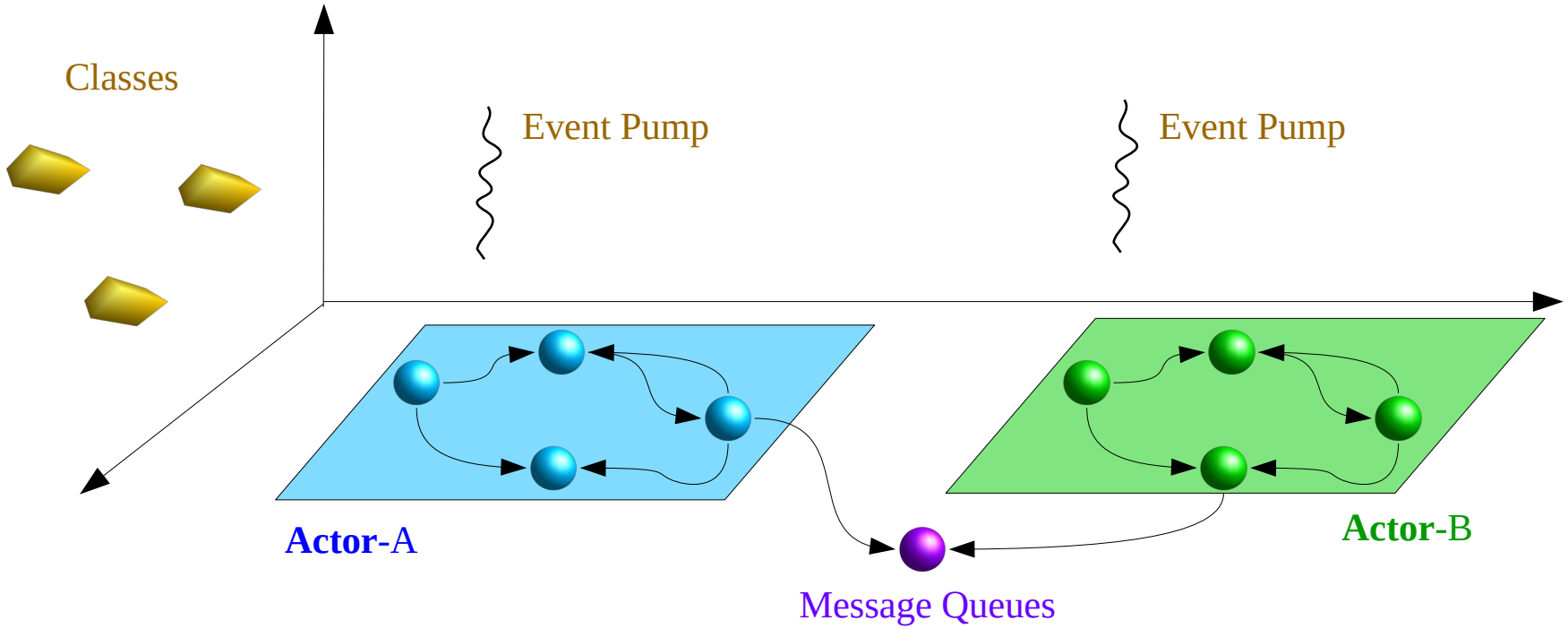
Discussing Fail-Stop Programming

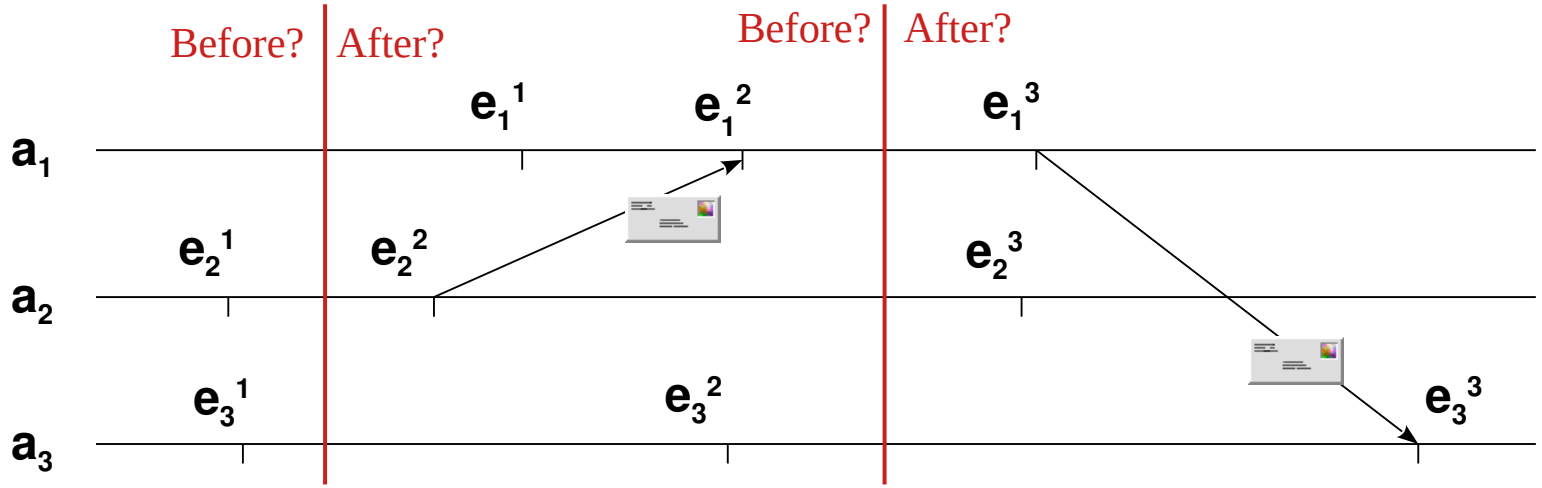
- It is the developers' job to ensure a fail-stop behavior
  - Your coding must ensure that there are no byzantine behaviors
  - This means no bugs, including proper exception handling
  - This also means no viruses…
- Suicide & Reincarnation
  - In doubt, a process must commit suicide and reincarnate itself
  - Usually requires a recovery period, once reincarnated



©Pr. Olivier Gruber

Discussing Time…





©Pr. Olivier Gruber

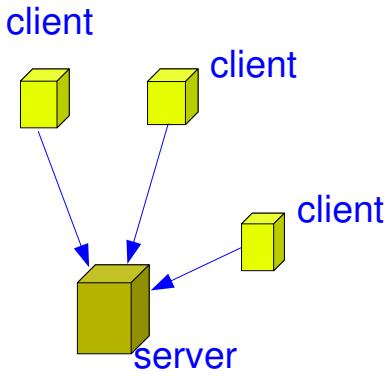
Keeping Track of Time

- Real hardware timers
  - Ticks a certain number of times per seconds
  - Time is the number of ticks since a certain known date
    - Like January first, 1970 for most Unix systems
- Clock skew
  - 60Hz timers do not tick exactly 60 per seconds
    - With modern chips, the skew is about 10^{-5}
    - Instead of 216,000 ticks per hour, we get between 215,998 and 216,002 ticks
- Clock synchronization
  - Several clocks therefore need to be synchronized
  - It can be done through different protocols
- The problem
  - *By the time you tell time… It is no longer the correct time...*

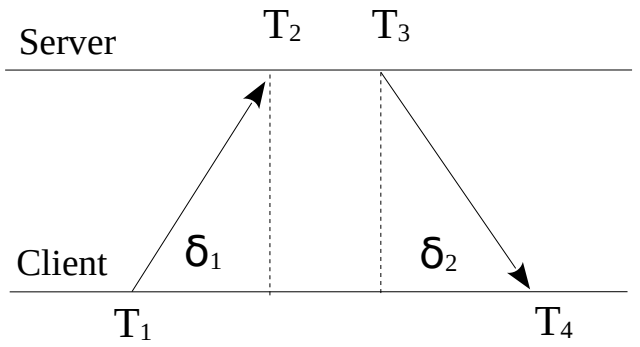
©Pr. Olivier Gruber

Network Time Protocol

- Cristian algorithm (1989)
  - Use a time server (with a correct UTC)
  - Takes into account message delays
- The problem?
  - What meaning can you give to (T_2-T_1) or (T_4-T_3) ?
  - Remember, clocks are not yet synchronized…



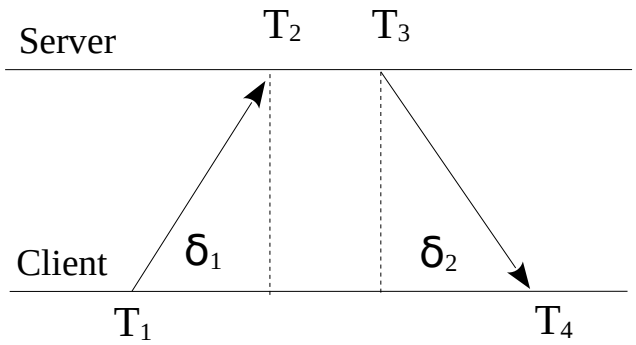
Any idea?



©Pr. Olivier Gruber

Network Time Protocol

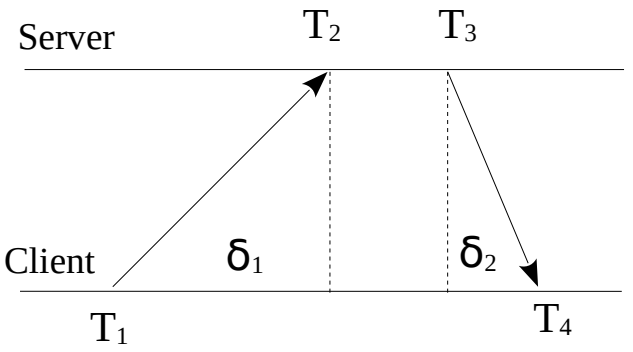
- How do we estimate what T4 should be?
  - The correction should be Θ such as
    - T4 + Θ = T3 + δ2
  - But all times Ti are local times, not synchronized
    - So δ2=(T4-T3) is meaningless
- Cristian algorithm (1989)
  - We use transmission delays
    - δ1=(T2-T1) δ2=(T4-T3)
  - We assume delays to be roughly constants
    - δ1 ≃ δ2
    - δ = (δ1 + δ2)/2
  - Compute time correction Θ
    - T4 + Θ = T3 + δ



©Pr. Olivier Gruber

Network Time Protocol

- Gradual change
  - Correction Θ can be negative or positive
    - Time can't go backward
    - Time should avoid leaps
  - This means you slow down or speed up your clocks
    - Each interrupt is either 9ms or 11 ms instead of 10ms
- Error correction
  - What if δ1 and δ2 differ too much...
    - Average Θs over multiple requests
    - Use multiple time servers and average Θs



©Pr. Olivier Gruber

Discussing Time

- Real time... is just an illusion
  - Literally speaking if we consider Einstein's relativity theory
  - May be precise enough in some situations, like for humans
  - But there will always be some margin for errors
- Avoid relying on the concept of a global time
  - It cannot be used to reason about a distributed system
  - It cannot be the basis of behavioral proofs
- If you still do use time...
  - You must be able to determine the maximum clock drift
  - Or allow for any drift

©Pr. Olivier Gruber

Group Broadcasting

- Assumptions
  - We assume a **static group of actors**
  - So each actor can communicate with the others in the group
- Group broadcasting without failures
  - A very simple and straightforward implementation
  - Just a simple function that sends a message to all the members of the group
- Group broadcasting with failures
  - What are the impacts of failures?

©Pr. Olivier Gruber

Reliable Group Broadcast – Specification

- A foundation mechanism for atomicity in distributed systems
  - Over a static or dynamic group of actors
- Reliable Broadcast
  - **Broadcasting**: a member Mi broadcasts a message to all members Mj, including itself (i=j)
  - **Atomicity**: all correct members eventually deliver a broadcasted message or none of them do
  - Important point: nothing is said about the order of delivery

Eventually: Delivery will happen in finite time

©Pr. Olivier Gruber

Reliable Broadcast – Design

- Protocol

Broadcast a message, noted *m*

Timestamp m with a sequence number, noted *seq(m)*

Identify sender, noted *sender(m)*

Send m to all members, including *sender(m)*

Receive a broadcasted message at a member M_i

Receive the message m (from the communication channel)

If the message has been delivered, just drop it

If this is the first time M_i receive m and sender(m) is not M_i

Send m to all members (but member M_i)

Deliver message m

Convinced?

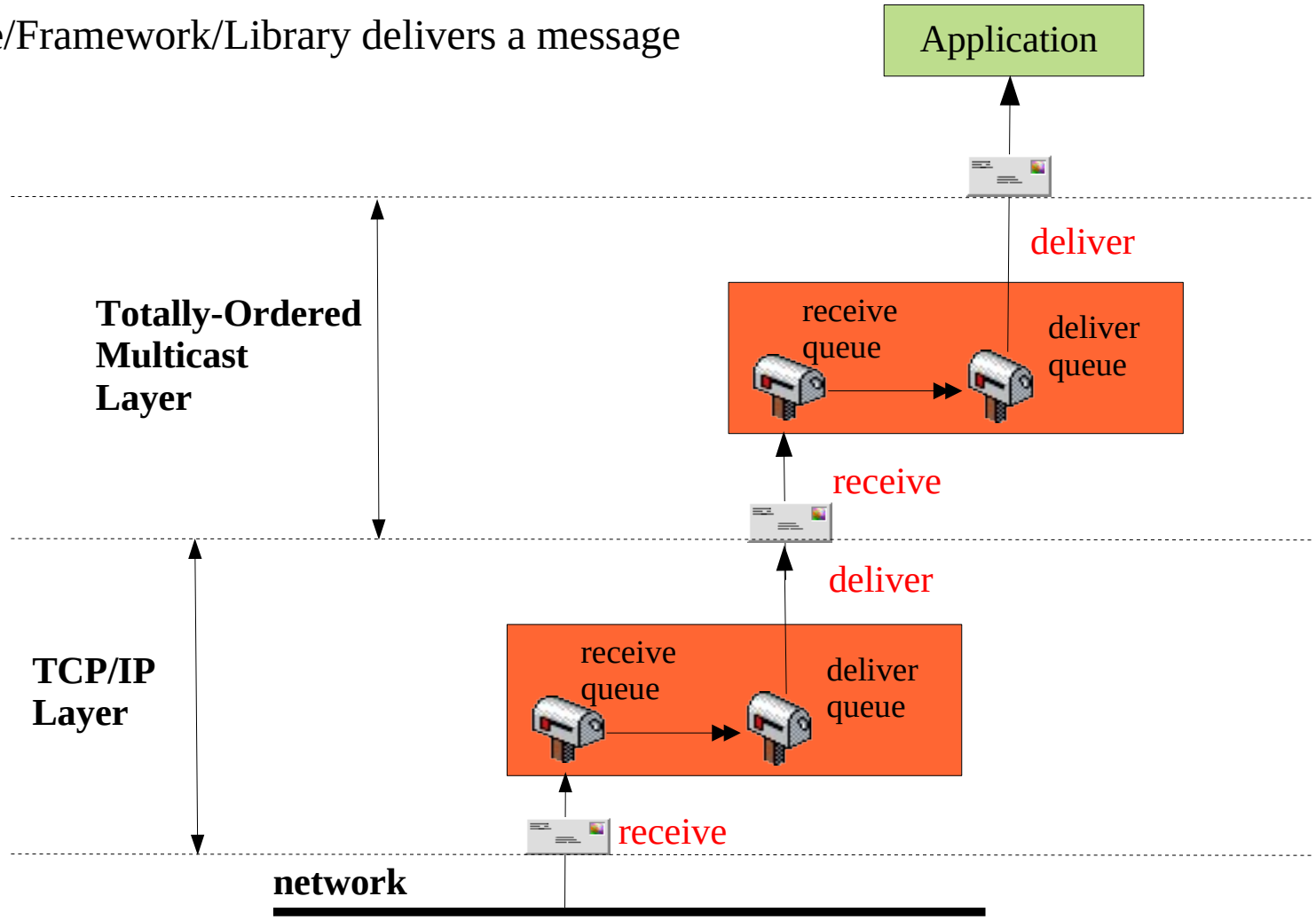
Note: This protocol is not optimized, better protocols exist, but this protocol shows it is possible to achieve a reliable broadcast under our assumptions. For more information:

V. Hadzilacos , S. Toueg, Fault-Tolerant Broadcast and Related Problems, in S. Mullender (ed.), *Distributed Systems* (2nd edition), Addison-Wesley, 1993

©Pr. Olivier Gruber

Import Distinction / Terminology

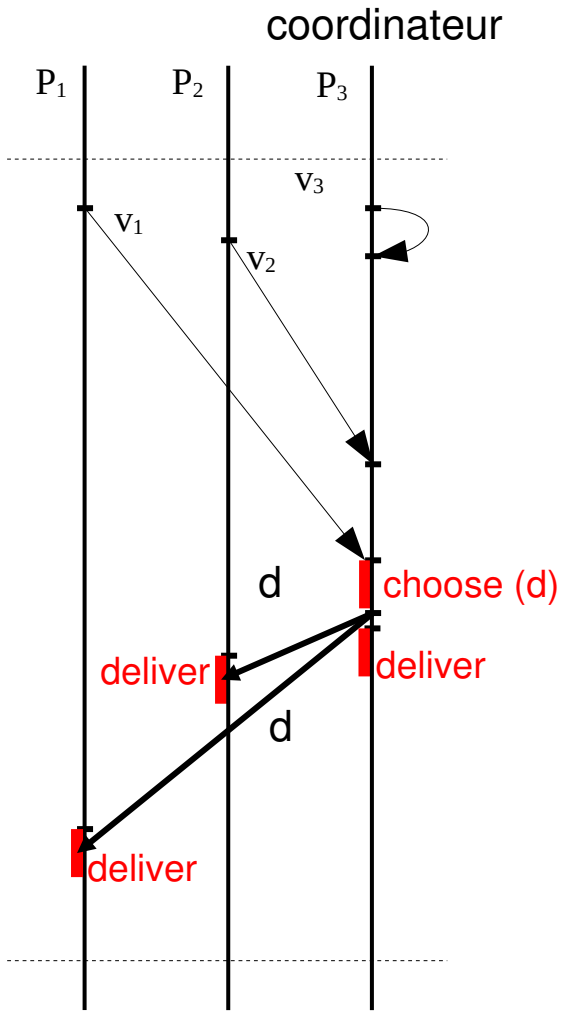
- **Receive versus Deliver**
  - Middleware/Framework/Library receives a message
  - Middleware/Framework/Library delivers a message



©Pr. Olivier Gruber

Coordinator-based Consensus

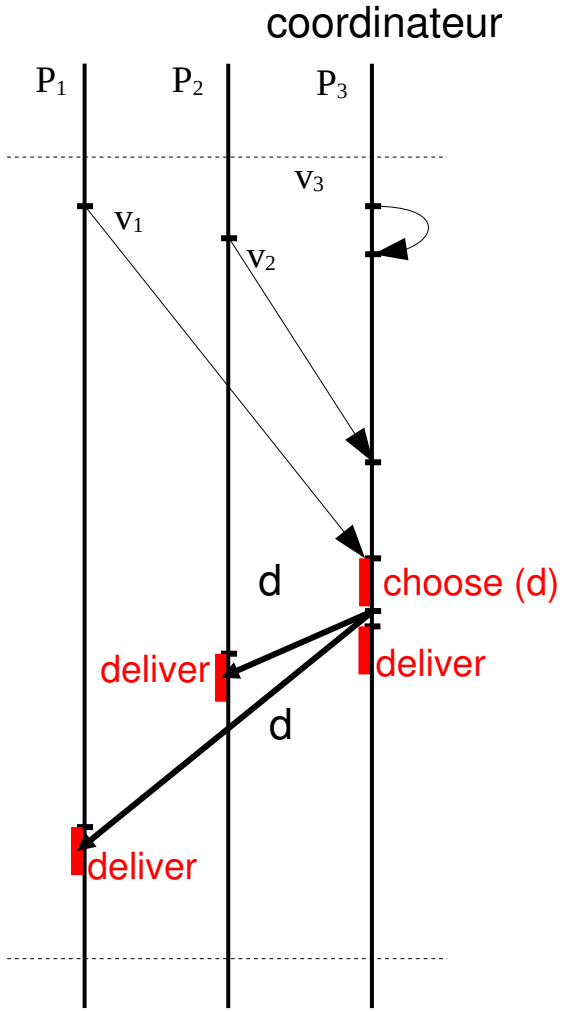
- Coordinator-based Solution, without failures
  - Each process sends its value to the coordinator
  - When the coordinator has all the values for the current period
  - Picks one of the proposed value
  - Sends that value to all processes
- Design
  - Easy enough to code…



©Pr. Olivier Gruber

Coordinator-based Consensus

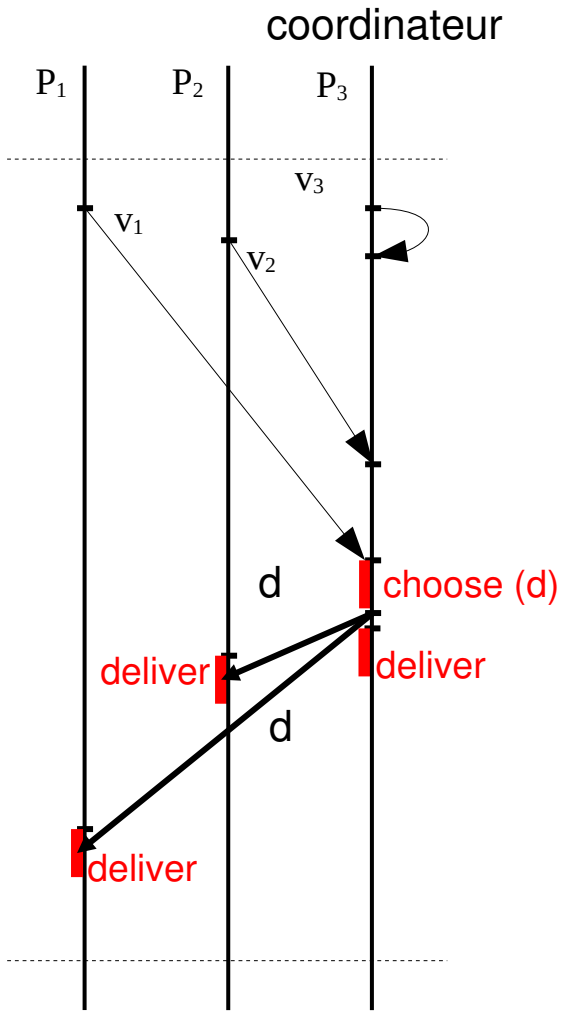
- Coordinator-based Solution, without failures
  - Each process sends its value to the coordinator
  - When the coordinator has all the values for the current period
  - Picks one of the proposed value
  - Sends that value to all processes
- Question
  - Easy enough to code… but harder to use
  - ***How/when can this consensus be used?***



©Pr. Olivier Gruber

Coordinator-based Consensus

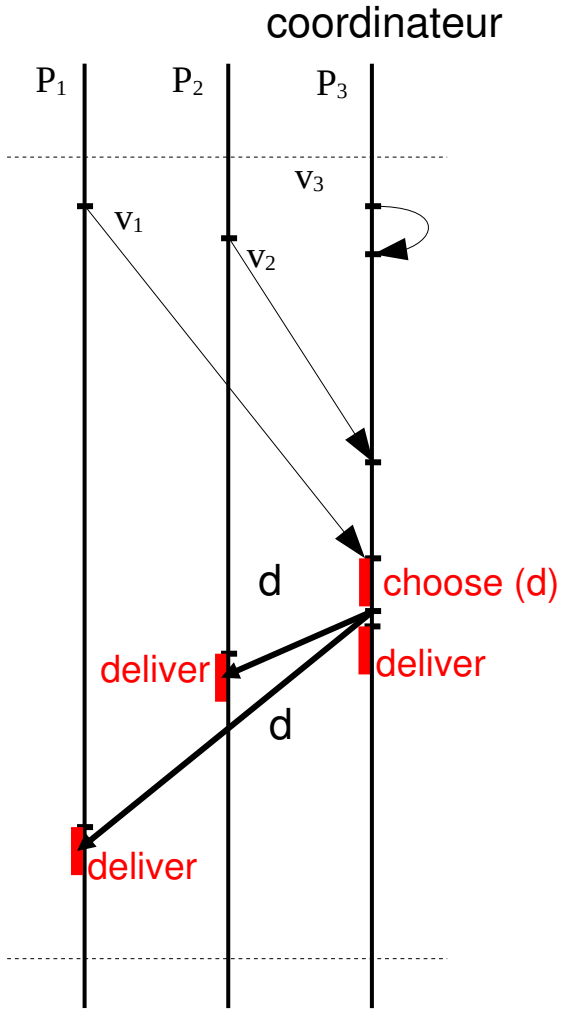
- Coordinator-based Solution, without failures
  - Each process sends its value to the coordinator
  - When the coordinator has all the values for the current period
  - Picks one of the proposed value
  - Sends that value to all processes
- Questions
  - When should a process send its value?
  - Does that require that clocks are synchronized?
  - Overlapping consensus?



©Pr. Olivier Gruber

Periodic Consensus

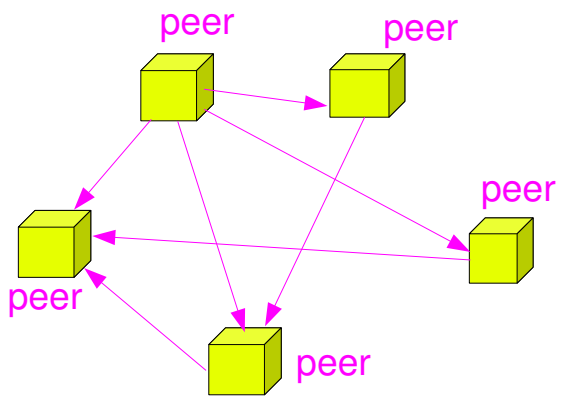
- Coordinator-based Solution, without failures
  - Periodically, each process sends its value to the coordinator
  - When the coordinator has all the values for the current period
    - Picks one of the proposed value
    - Sends that value to all processes
- Design points
  - Does having a periodic design help?
  - What about the period? Long? Short?
  - What about failures?

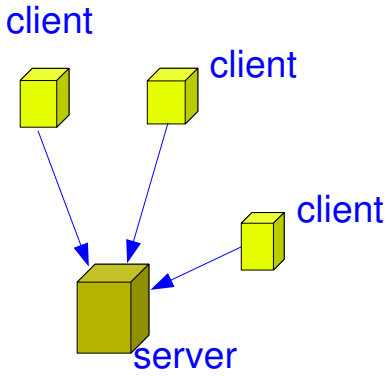


©Pr. Olivier Gruber

Distributed Mutual Exclusion

- Mutual exclusion
  - Only one process in a critical section at any given time
  - Critical section is granted until released
- Data consistency
  - When granted a critical section, the protected data must be up-to-date
- Possible architectures
  - Client-Server architecture
  - Peer-to-Peer architecture

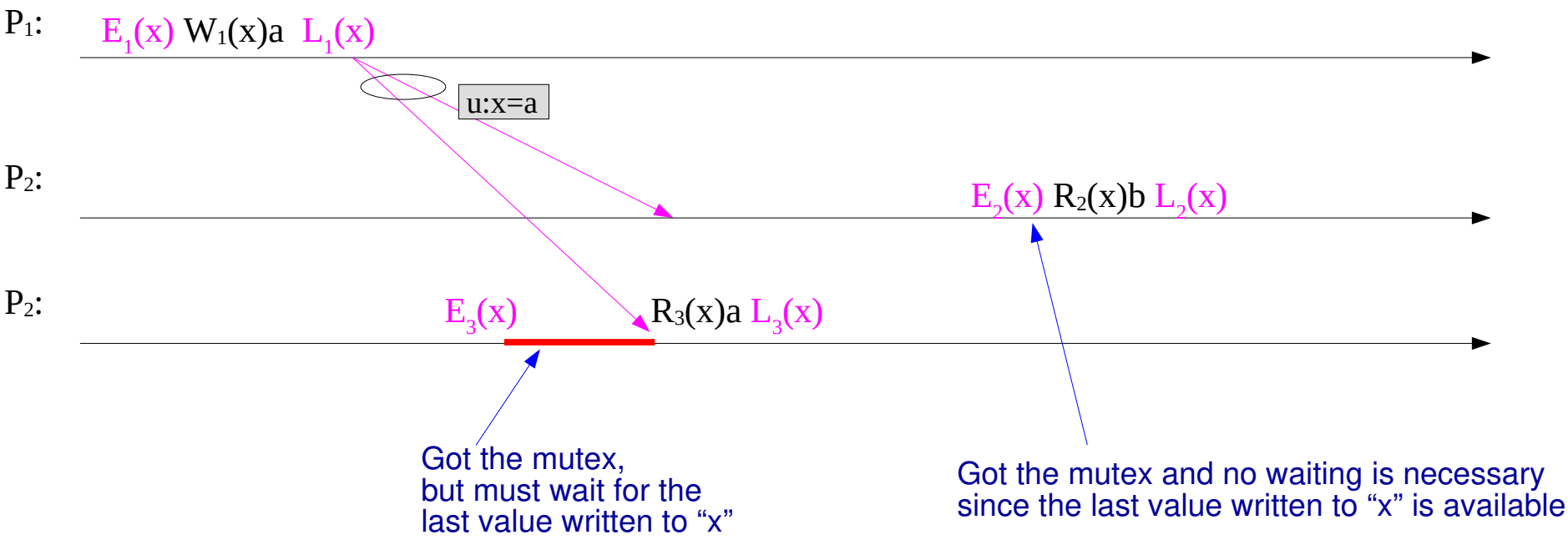




©Pr. Olivier Gruber

Mutex & Data Consistency – Eager Approach

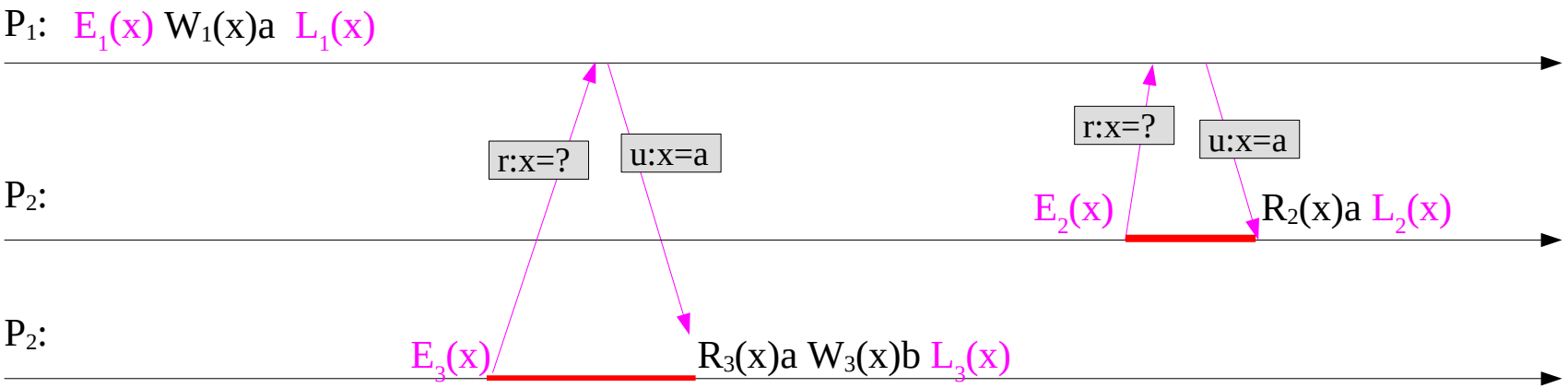
- Eager Approach
  - When leaving the critical section on “x”, *eagerly* ***group broadcast*** the latest value of “x”
  - The idea is that most of the time, the last written values will be available without waiting



©Pr. Olivier Gruber

Mutex & Data Consistency – Lazy Approach

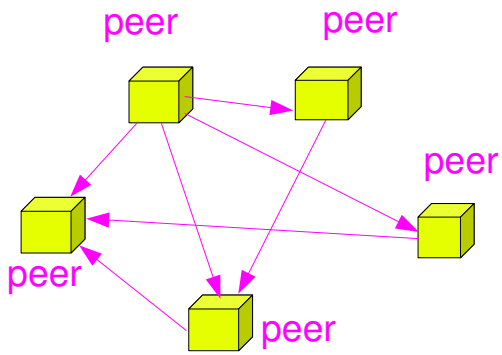
- Lazy Approach
  - Upon entering a critical section on “x”, *lazy* request the latest written value for “x”
  - The request must be made to the process that was last granted the critical section
  - The idea is to avoid broadcasting updates too often and to peers who do not need them

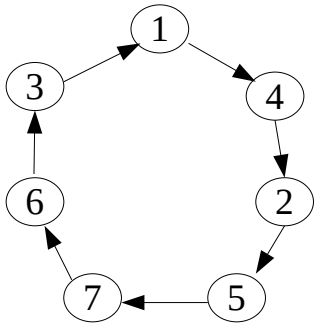


©Pr. Olivier Gruber

Distributed Mutual Exclusion on a Ring Overlay

- Design
  - Overlay ring, no matters what the real network topology is
  - There is one token per resource, going around the ring
  - A site can only enter the critical section if it has the token
  - A site forwards the token when leaving the critical section
- Classwork
  - What about peer failures?
  - What about data consistency?

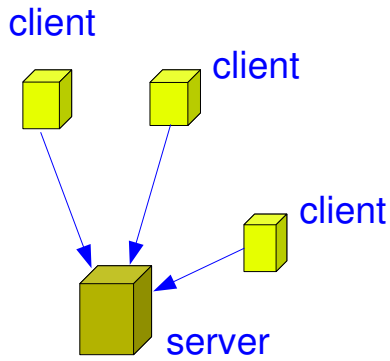


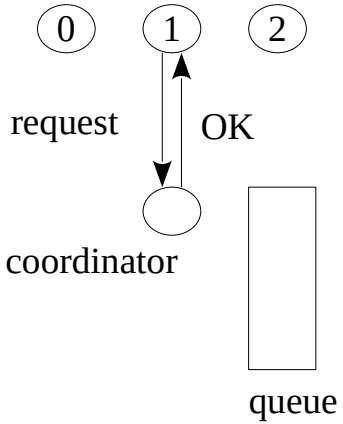


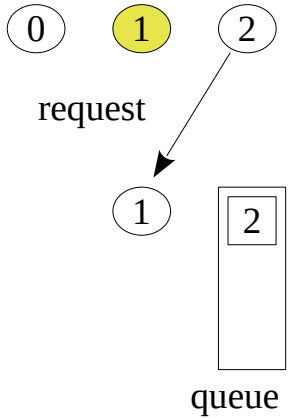
©Pr. Olivier Gruber

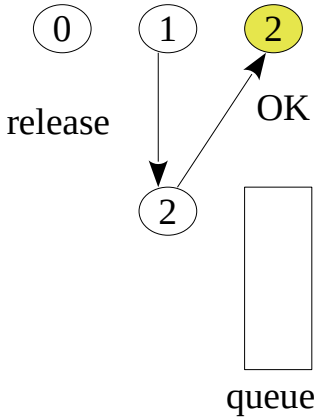
Distributed Mutual Exclusion

- Coordinator approach
  - The coordinator grants the critical section if available
  - When not available, it queues the requesting processes
  - When critical section is freed, it schedules the first process in the queue









©Pr. Olivier Gruber

Classwork – Fault-tolerant Distributed Mutual Exclusion

- Client-server architecture
  - Each mutex protects one data item: mutex_enter(data-id) / mutex_leave(data-id)
  - Client processes run an application and a server-aware library to acquire/release mutexes
  - The server manages the mutexes and the protected data items
- Classwork
  - Design fault-tolerance: clients may fail, the server may fail
  - Assume a perfect failure detector
- Advice
  - Divide and conquer…
  - Do not try to do everything at once...

©Pr. Olivier Gruber